

Valeurs d'adhérence de la suite logistique

Conseil : Pour ce TP, préférer EduPython à Spyder (graphes de meilleure qualité).

On s'intéresse à la suite logistique $(u_n)_{n \in \mathbb{N}}$ définie par :

$$u_0 = 0.5, \quad \forall n \in \mathbb{N}, \quad u_{n+1} = r \cdot u_n(1 - u_n), \quad \text{où } r > 0 \text{ est une constante.}$$

Cette suite modélise l'évolution d'une population dans un environnement aux ressources limitées :

- u_n représente la population (relative, entre 0 et 1) présente à la génération n .
- Le réel $r > 0$ mesure le taux de reproduction.
- Le terme en " $x(1-x)$ " traduit une compétition interne des individus pour les ressources : si u_n est trop grand (proche de 1), à la génération suivante on observera une forte baisse de la population...

Ce modèle déterministe et extrêmement simple produire des comportements inattendus. Selon la valeur du paramètre r , on observera : une stabilisation de la population sur une valeur donnée, des oscillations périodiques entre différentes valeurs, ou encore un comportement chaotique imprévisible à long terme.

✎ Exercice 1

1. Compléter la fonction suivante pour que, à la donnée d'un réel r et d'un entier n , elle affiche sur une figure les termes $u_0, u_1, \dots, u_n$.

```
import numpy as np
import matplotlib.pyplot as plt

def afficher(r,n) :

    X = ..... ; Y = ..... ; u = ....

    for i in range( ..... ) :
        Y[i] = u

        u = .....

    plt.plot(X,Y, '.-' )
    plt.show()
```

2. Constater graphiquement les comportements suivants :

- Pour $0 < r \leq 1$, la population **s'éteint**. (Tester $r = 0.5, r = 0.8, r = 1, \dots$)
- Pour $1 \leq r \leq 3$, la population **se stabilise** sur une valeur strictement positive. (Tester $r = 1.5, r = 2, r = 2.5, r = 2.8$. Pour $r = 3$, la convergence est très lente)
- Pour $3 < r \leq 3.45$, la population oscille asymptotiquement entre 2 valeurs. On dit qu'elle admet **2 valeurs d'adhérence**. (Tester $r = 3.05, r = 3.2, r = 3.4$)

- Pour $3.5 \leq r \leq 3.53$, la population oscille asymptotiquement entre 4 valeurs. On dit qu'elle admet **4 valeurs d'adhérence**. (Tester $r = 3.5, r = 3.52$)

Pour y voir plus clair, on pourra remplacer l'option graphique `'.-'` par `'.'`

3. On voudrait un programme qui, à la donnée d'un paramètre r , affiche sur un graphique les valeurs d'adhérence de la suite $(u_n)_{n \in \mathbb{N}}$ correspondante. Pour cela :

- On calcule les termes de la suite jusqu'à u_{1000} .
- On affiche, sur un même graphe, les 200 points :

$$(r, u_{1000+1}), (r, u_{1000+2}), (r, u_{1000+3}), \dots, (r, u_{1000+200}).$$

(Autrement dit, toutes ces valeurs sont affichées à l'abscisse r)

```
def adherence(r) :

    M = 1000; n = 200;

    X = ..... ; Y = ..... ; u = ....

    for i in range( ..... ) :

        u = r * u * (1-u)

        if ..... :

            Y[i-M] = u

    plt.plot(X,Y, '.k')
```

4. Tester la fonction `adherence` (dans la console : `adherence(r); plt.show()`) et vérifier qu'on obtient bien une seule valeur d'adhérence pour $1 \leq r \leq 3$, deux valeurs pour $3 < r \leq 3.45$, quatre valeurs pour $3.5 \leq r \leq 3.53$.

Déterminer une valeur de r avec 8 valeurs d'adhérence : $r = \dots\dots\dots$

5. Compléter pour finir la fonction suivante pour qu'elle affiche le **diagramme de bifurcation** de la suite logistique, c'est à dire pour qu'elle affiche, en fonction de l'abscisse $r \in [0, 4]$, les valeurs d'adhérence de la suite logistique associée.

(Il s'agit d'une fonction "multivaluée" : à une abscisse on associe plusieurs ordonnées!)

```
def bifurcation()

    R = .....

    for r in R :

        adherence(r)

    plt.show()
```

Pour rendre le graphe plus fin, dans la fonction "`adherence`", on remplacera :

`plt.plot(X,Y, '.k')` par `plt.plot(X,Y, ',k')`.



Exercice 2

Théorie du chaos (Sensibilité aux conditions initiales)

On considère deux suites avec des données initiales légèrement différentes :

$$u_0 = 0.50001 \text{ et } \forall n \in \mathbb{N}, u_{n+1} = r \cdot u_n(1 - u_n),$$

$$v_0 = 0.50002 \text{ et } \forall n \in \mathbb{N}, v_{n+1} = r \cdot v_n(1 - v_n),$$

1. Compléter la fonction suivante pour que, à la donnée d'un réel r et d'un entier n , elle affiche sur une même figure les termes $(u_k)_{k \in \llbracket 0, n \rrbracket}$ et $(v_k)_{k \in \llbracket 0, n \rrbracket}$.

```

import numpy as np
import matplotlib.pyplot as plt

def comparer(r,n) :

    X = ..... ;

    U = ..... ; U[0] = .....

    V = ..... ; V[0] = .....

    for i in range( ..... ) :

        U[i] = .....

        V[i] = .....

    plt.plot( ... , ... ) ; plt.plot( ... , ... )
    plt.show()

```

2. Comparer graphiquement les 50 premiers termes de u et v lorsque :

$$r = 1.5, \quad r = 3.2, \quad r = 3.5$$

On constate que le comportement des deux suites est sensiblement le même : une petite imprécision dans la donnée initiale ne produit pas de changement drastique.

Comparer à présent les 50 (et 100, et 200) premiers termes de u et v lorsque $r = 4$.

La suite logistique est un célèbre exemple de **système chaotique**. Pour certaines valeurs du paramètre r , deux populations dont les effectifs initiaux ne diffèrent que d'un millionième peuvent évoluer de façon complètement différente après quelques dizaines de générations. Le hasard n'intervient pourtant nulle part : l'imprévisibilité provient uniquement de l'extrême **sensibilité aux conditions initiales** !